

Cognitive Kernel v1.0 — Engineering Specification

Wolf-CoG-OS / AAIS Unified Cognitive Layer

Status: Draft-Stable

Author: Jon Halstead

Reviewed by: Dar-z Morris

Version: 1.0.0

Scope: Kernel-level cognitive governance and runtime separation

0. Overview

The Cognitive Kernel (CK) defines the governed cognition pipeline for Wolf-CoG-OS.

It enforces deterministic classification, runtime separation, bounded synthesis, and replayable decision state across all cognitive operations.

The kernel is composed of:

Substrate Classifier (ULS)

Hysteresis Engine

Six Cognitive Runtimes

Questioner Runtime

Answerer Runtime

Deliberation Runtime

Reflection Runtime

Execution Runtime

Memory Runtime

Kernel Orchestrator

Thread State Store

All cognition flows through this pipeline.

No runtime may bypass substrate classification or hysteresis authority.

1. Kernel Invariants

CK-I1 — Deterministic First Contact

Every incoming message **MUST** be processed by the Substrate Classifier before any runtime is invoked.

CK-I2 — Runtime Separation

No runtime may perform the responsibilities of another.

No runtime may self-promote or self-demote.

CK-I3 — Hysteresis Authority

Only the Hysteresis Engine may change the active runtime.

CK-I4 — Replayability

All decision-relevant state **MUST** be reconstructable from logs.

CK-I5 — Bounded Synthesis

Generative components **MUST NOT** fabricate missing context.

CK-I6 — Explicit Uncertainty

Ambiguity **MUST** be surfaced, not hidden.

CK-I7 — Deterministic Switch Rationale

Every runtime switch **MUST** include a machine-readable rationale.

2. Substrate Layer (ULS)

2.1 Input

Code

{

text: string,

prior_question_count: int,

thread_metadata?: object

```
}
```

2.2 Feature Extraction

Deterministic counters + regex + math:

pronoun_ambiguity

vague_noun_density

hedge_density

knowledge_gap_signal

meta_intent_signal

subject_specificity

context_saturation

prior_question_count

lexical_density

domain_specificity

2.3 Intent Vector

Weighted linear combination →

Code

```
{
```

questioner_score: float,

answerer_score: float,

probe_score: float,

handoff_score: float

```
}
```

2.4 Ambiguity Class

Code

NONE | SURFACE | STRUCTURAL | EPISTEMIC | TERMINAL

2.5 Substrate Intent

Code

```
{  
  
mode: QUESTIONER | ANSWERER | DELIBERATION | REFLECTION | EXECUTION |  
HANDOFF | NONE,  
  
confidence: float,  
  
ambiguity_class: enum,  
  
rationale: string,  
  
suggested_probes?: [QuestionSkeleton]  
}
```

3. Hysteresis Engine

3.1 State

Code

```
{  
  
current_mode,  
  
last_mode,  
  
consecutive_same_mode_count,  
  
switch_count,  
  
context_saturation,  
  
force_lock?: boolean  
}
```

3.2 Switch Rules

Switch allowed immediately if:

intent.force_switch == true

mode == HANDOFF

context_saturation $\geq$ 0.80 (Q→A)

human override phrase detected (“just answer”, “go ahead”, “proceed”)

Otherwise require:

N consecutive identical SubstrateIntent.mode signals (default N=2)

3.3 Output

Code

```
runtime.switch {  
  from: mode,  
  to: mode,  
  rationale: string,  
  hysteresis_state: object  
}
```

4. Runtime Definitions

Below are the engineering-grade contracts for each runtime.

4.1 Questioner Runtime

Role

Ambiguity decomposition and information-gain maximization.

Input

SubstrateIntent

message

QuestionerContext

Process

Generate QuestionSkeleton objects from deterministic rules.

Call LLM ONLY to naturalize phrasing.

Rank by info_gain.

Stream questions.

Output Events

Code

questioner.start

questioner.question {type, skeleton_id, text}

questioner.context {pending, answered, saturation}

questioner.complete

Invariants

MUST NOT introduce new semantic content.

MUST NOT answer anything.

MUST stop when context_saturation $\geq$ threshold.

4.2 Answerer Runtime

Role

Bounded synthesis over validated context.

Strategy Selection

Code

DIRECT if answerer_score > 0.7 and no Q-context

CONTEXTUAL if answerer_score > 0.8 and Q-context exists

BOUNDED if moderate score or residual ambiguity

DEFERRED if confidence < 0.25 and no context

Output Events

Code

answerer.start {strategy, has_context}

answerer.chunk "..."

answerer.complete {strategy}

Invariants

MUST declare strategy.

MUST name gaps in BOUNDED/DEFERRED.

MUST NOT fabricate missing context.

4.3 Deliberation Runtime

Role

Multi-option reasoning + tradeoff analysis.

Core Object: DecisionFrame

Code

{

options: [Option],

criteria: [Criterion],

scores: {option_id: float},

tradeoffs: [Tradeoff],

dominant_option: option_id | null,

confidence: float

}

Invariants

MUST produce ≥ 3 options unless constrained.

MUST expose scoring function.

MUST NOT narrativize until scoring is complete.

4.4 Reflection Runtime

Role

Self-audit and coherence evaluation.

ReflectionReport

Code

```
{  
  coherence_score: float,  
  assumptions: [Assumption],  
  risk_factors: [Risk],  
  missing_context: [Gap],  
  revision_suggestions: [Suggestion]  
}
```

Invariants

MUST NOT modify original output.

MUST NOT introduce new domain facts.

MUST run before high-stakes outputs.

4.5 Execution Runtime

Role

Plan $\rightarrow$ Act $\rightarrow$ Verify loop.

ExecutionPlan

Code

```
{  
  steps: [Step],  
  preconditions: [Condition],  
}
```



```
postconditions: [Condition],  
failure_modes: [FailureMode],  
rollback_strategy: RollbackPlan  
}
```

Invariants

MUST verify preconditions before each step.

MUST halt on uncertainty.

MUST emit receipts for each step.

4.6 Memory Runtime

Role

Long-scale continuity and retrieval.

MemoryNode

Code

```
{  
id,  
type: episodic | semantic | procedural | governance,  
payload,  
parents: [id],  
timestamp  
}
```

Invariants

MUST store structured facts only.

MUST maintain causal traceability.

MUST support deterministic replay.

5. Kernel Orchestrator

Responsibilities

Maintain thread state

Route messages to correct runtime

Apply hysteresis decisions

Emit SSE events

Persist turn logs

Turn Pipeline

Code

Incoming message

→ Substrate Classifier

→ SubstrateIntent

→ HysteresisEngine

→ Runtime selection

→ Runtime execution

→ MemoryRuntime.store

→ TurnRecord.persist

6. Thread State Model

ThreadState

Code

{

thread_id,

active_runtime,

hysteresis_state,

```
context_saturation,  
questioner_context,  
decision_frames,  
reflection_reports,  
execution_plans,  
memory_handles,  
turn_log: [TurnRecord]  
}
```

TurnRecord

Code

```
{  
turn_id,  
timestamp,  
input_message,  
substrate_signal,  
intent_vector,  
substrate_intent,  
active_runtime,  
runtime_events: [Event],  
switch_rationale,  
summary_outcome  
}
```

7. Failure Modes

FM-1: Substrate Misclassification

Detected by Reflection or human

Action: mark turn, reclassify, log

FM-2: Mode Flapping

Action: increase hysteresis threshold, temporary lock

FM-3: Overconfident Answerer

Action: downgrade to BOUNDED/DEFERRED

FM-4: Execution Drift

Action: halt, reflect, deliberate, log governance event

FM-5: Memory Corruption

Action: quarantine nodes, mark thread tainted

8. Interfaces

REST

POST /uls/thread

GET /uls/thread/{id}

POST /uls/thread/{id}/turn

POST /uls/thread/{id}/answer

DELETE /uls/thread/{id}

GET /uls/threads

POST /uls/classify

SSE Events

uls.classified

runtime.switch

questioner.*

answerer.*

deliberation.*

reflection.*

execution.*

memory.*

uls.turn_complete

9. Implementation Notes

Runtimes are independent modules under operator_kernel/uls/.

Orchestrator owns thread lifecycle.

Substrate classifier MUST remain pure logic.

Brain (LLM) is a pluggable dependency with strict contracts.

Memory runtime MUST be append-only with causal links